OS LAB ASSIGNMENT-4

GROUP-18:
Vislavath Bindhu (230101112)
Tippana Likitha Reddy (230101105)
Bommagani Rohitha (230101027)
Annadevara Dedeepya Siri (230101011Drive link for deliverables:https://drive.google.com/drive/folders/1iTFdykuFnfGfpeu_h_q7uIzot353dUR1?usp=sharing

Task 1: COW implementation

1. Overview

Copy-On-Write (COW) is a memory optimization technique in which pages are shared between parent and child after fork until a write occurs. The kernel keeps track of reference counts for each physical page and allocates a new page only when a write occurs, minimizing memory usage.

This implementation in xv6 involves modifications in:

- Memory allocator (kalloc.c)
- Virtual memory (vm.c)
- Trap handling (usertrap.c)
- User-level tests (cowtest.c)

2. Kernel Changes

2.1) kernel/kalloc.c

1. Reference Count Array and Lock

```
// Reference count for each physical page
int refcnt[PHYSTOP/PGSIZE];
struct spinlock ref_lock;
```

Reason:

- To track how many processes share a physical page.
- Essential for COW: ensures that pages are only freed when no process references them.
- ref_lock ensures atomic updates in multi-core or preemptive environments. 2.

  Modified kalloc()

```
void* kalloc(void) {
    struct run *r;

    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r)
        kmem.freelist = r->next;
    release(&kmem.lock);

    if(r){
        memset((char*)r, 5, PGSIZE); // fill with junk
        acquire(&ref_lock);
        refcnt[(uint64)r/PGSIZE] = 1; // initialize refcount
        release(&ref_lock);
        return (void*)r;
    }

    return 0; // no free pages
}
```

Reason:

- Allocated pages now start with refcnt = 1.
- Required for COW so the kernel knows initially only one process owns the page.
- Junk filling (memset) helps catch dangling references during debugging.

3. Modified kfree()

```
void kfree(void *pa) {
    acquire(&ref_lock);
    if(--refcnt[(uint64)pa/PGSIZE] > 0){
        release(&ref_lock);
        return; // still shared, don't free
    }
    release(&ref_lock);

    memset(pa, 1, PGSIZE); // fill with junk

    struct run *r = (struct run*)pa;
    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
```

Reason:

- A page is freed only if refcnt reaches 0, preventing freeing pages still shared by other processes.
- Maintains safe memory sharing for COW.
- Fills freed memory with junk to detect dangling pointers or illegal accesses.

4. Reference Count Helper Functions

```
void kref_inc(uint64 pa) {
    acquire(&ref_lock);
    refcnt[pa/PGSIZE]++;
    release(&ref_lock);
}

int kref_dec(uint64 pa) {
    acquire(&ref_lock);
    int x = --refcnt[pa/PGSIZE];
    release(&ref_lock);
    return x;
}

int kref_get(uint64 pa) {
    acquire(&ref_lock);
    int x = refcnt[pa/PGSIZE];
    release(&ref_lock);
    return x;
}
```

Reason:

- Simplifies incrementing, decrementing, and reading the reference count.
- Ensures atomic operations on refcnt across processes.
- Used in fork, uvmcopy(), and COW page fault handling.

2.2)kernel/trap.c

1)trap.c → usertrap()

```
else if(r_scause() == 13 || r_scause() == 15){
    // Load (13) or Store (15) page fault
    uint64 va = PGROUNDDOWN(r_stval());

    // Handle copy-on-write
    if(cowfault(p->pagetable, va) < 0){
        // Invalid access
        printf("usertrap: invalid page fault pid=%d va=0x%lx\n", p->pid, va);
        setkilled(p);
    } else {
        p->pstats.page_faults++;
    }
}
```

Reason:

- Detects page faults caused by writing to shared pages.
- Calls cowfault() to allocate a private copy for the writing process.
- Increments page fault statistics (pstats.page_faults).
- Avoids process crash if COW is possible.

2.3)kernel/vm.c

1)Replaced original uvmcopy which physically copied all pages, with a COW-aware version:

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
  pte_t *pte;
  uint64 pa, i;
  uint flags;

  for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0)
      panic("uvmcopy: pte should exist");
    if((*pte & PTE_V) == 0)
      panic("uvmcopy: page not present");
    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);
    *pte &= ~PTE_W;
    if(mappages(new, i, PGSIZE, pa, (flags & ~PTE_W)) != 0){
      uvmunmap(new, 0, i/PGSIZE, 1);
      return -1;
    }
    kref_inc(pa);
  }
  return 0;
}
```

2)COW Page Fault Handler (cowfault)

```
int cowfault(pagetable_t pagetable, uint64 va){
  pte_t *pte; uint64 pa; uint flags; char *mem;
  va = PGROUNDDOWN(va);
  if(va >= MAXVA) return -1;
  if((pte = walk(pagetable, va, 0)) == 0) return -1;
  if((*pte & PTE_V) == 0 || (*pte & PTE_U) == 0) return -1;
  pa = PTE2PA(*pte); flags = PTE_FLAGS(*pte);
  if(flags & PTE_W) return 0;
  if(kref_get(pa) > 1){
    if((mem = kalloc()) == 0) return -1;
    memmove(mem, (char*)pa, PGSIZE);
    kref_dec(pa);
    *pte = PA2PTE(mem) | PTE_FLAGS(*pte) | PTE_W;
  } else *pte |= PTE_W;
  return 0;
}
```

Reason:

- Handles write faults on COW pages.
- Allocates a new page only when multiple processes share it.
- Preserves the original page for other processes.
- Efficient: no unnecessary copying for single-owner pages.

2.4)kernel/defs.h

Added prototypes for reference-count helpers:

```
void kref_inc(uint64 pa);
int  kref_dec(uint64 pa);
int  kref_get(uint64 pa);
```

Reason:

- Needed for Copy-On-Write (COW) implementation.
- kref_inc increments the reference count for shared pages.
- kref_dec decrements the reference count when a page is freed or replaced.
- kref_get reads the reference count to decide if a page should be copied on write.

2. VM / Paging Changes

Added COW page-fault handler prototype:

```
int cowfault(pagetable_t pagetable, uint64 va);
```

Reason:

- Allows trap.c to call a COW fault handler during a page fault.
- Supports lazy page copying when a write occurs to a shared page.

2.5)kernel/proc.h

```
// Statistics per-process
struct pagestat {
  uint64 page_faults; // total page faults
};

// In struct proc
struct pagestat pstats;       // paging stats
```

Reason:

- To track per-process page faults caused by lazy allocation or COW.
- Useful for performance monitoring and debugging.
- Updated in usertrap() whenever a page fault is handled.

3)user changes 3.1)user/cowtest.c

Purpose:

- Tests the Copy-On-Write (COW) functionality implemented in vm.c and trap.c.
- Verifies that memory pages are shared read-only between parent and child after fork() until a write occurs.
- Ensures that page reference counting and cowfault() handling works correctly.

How cowtest Works

1. Fork a Child Process
   - Parent calls fork() to create a child.
   - Child inherits the parent's memory via COW, so pages are shared read-only.
2. Read from Shared Memory
   - Both parent and child read from inherited pages.

- Should not trigger a page fault, because read-only access is allowed.

3. Write to Shared Page
    - Either parent or child writes to the shared page.
    - Triggers a copy-on-write page fault, handled by cowfault().
    - A new physical page is allocated, and the write succeeds.
4. Verify Isolation
    - After the write, parent and child should have separate copies of the page.
    - Modifying one page should not affect the other.
5. Check Reference Counts
    - kref_get() verifies that page reference counts are correctly incremented/decremented.
    - Helps ensure no memory leaks or premature frees.

Expected Behavior / Output

- Initial fork: "Child and parent sharing pages."
- After write: "Copy-on-write page fault handled, new page allocated." Final verification:
- COW test passed: parent and child pages are independent If the implementation fails:
    - Writes could affect both parent and child. o Reference count mismanagement may lead to double free or memory leaks.

Implementation Notes

- Uses user-space memory allocations (malloc) for testing.
- Works with your modified uvmcopy() that marks pages read-only for COW.
- Relies on trap handler to catch write faults and call cowfault().

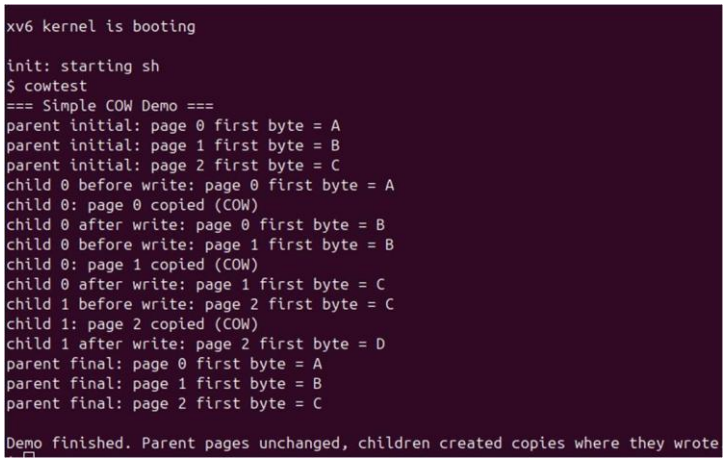
4)MAKEFILE

```
UPROGS = $U/_cat $U/_echo ... $U/_cowtest
```

Reason:

- Include COW test program and others for verification.
- Integrated into fs.img build.

Output



- The parent allocates 3 pages: page 0='A', page 1='B', page 2='C'.
- On fork(), children share parent pages read-only (COW).
- When a child writes to a shared page, a COW page fault occurs:
  - A new physical page is allocated.
  - Original data is copied.
  - Page marked writable for the child.

- Child 0 writes pages 0 and 1 → new copies created.
- Child 1 writes page 2 → new copy created.
- Parent pages remain unchanged.

Conclusion: COW works correctly: children get private copies on write, parent memory is preserved.

Task 2: Demand Paging with MRU Page Replacement

Objective:

The goal of this project was to implement demand paging in the xv6 operating system using a Most Recently Used (MRU) page replacement policy. This involves:

1. Maintaining a global MRU list of all resident user pages.
2. Swapping pages in/out on demand when memory is exhausted.
3. Adding system calls for monitoring page faults, swap-ins, swap-outs, and MRU ordering.
4. Demonstrating correctness through a user program (mrumem.c).

1. MRU Page Replacement Design

The MRU policy ensures that the most recently accessed page is the first to be evicted when physical memory is exhausted. This is implemented via:

- MRU List: A doubly-linked list (struct mru_entry) storing pid, vpn, and optionally the kernel virtual address of the page.

- Page Access Tracking: Every memory access (via page faults) updates the MRU list to mark the page as most recently used.
- Eviction: When memory is full, the head of the MRU list (most recently used page) is removed and swapped out to disk. 2. Kernel changes 2.1 proc.h:

```
struct proc* getproc_by_pid(int pid);

struct pagestat {
  int pagefaults;
  int swapins;
  int swapouts;
};
```

```
  struct pagestat pst;
  struct file *swapfile;     // per-process swap file (kernel file struct)
  uchar *swapmap;            // bitmap/array: one byte per vpn; 0=resident,1=swapped
  int swapped_pages;         // count swapped
```

- struct pagestat: Tracks per-process page faults, swap-ins, swap-outs.
- swapfile: File on disk to store evicted pages.
- swapmap: Bitmap to mark which virtual pages are swapped (1) or resident (0).
- swapped_pages: Number of pages currently swapped.
- Purpose: Stores metadata for demand paging and MRU tracking per process.

2.2. swap.c

- ensure_swapfile(p): Creates per-process swap file (/swap.<pid>) and swapmap if not already present.
- swap_out_page(p, va):
  - Writes the physical page at va to swap file.
  - Marks page as swapped in swapmap. o Frees the physical page and invalidates PTE.
  - Updates swapouts counter.
- swap_in_page(p, va):
  - Allocates a physical page and reads content from swap file. o Updates page table and MRU list.
  - Updates swapins counter.
- Purpose: Handles swapping in/out pages during demand paging.

The mru.c file implements the Most Recently Used (MRU) page replacement policy for demand paging. Its main responsibilities are maintaining the MRU list of all resident pages, updating page access order, and selecting pages for eviction when memory is full.
MRU List Structure:

- Represents a node in the MRU list.
- Stores which process owns the page and its virtual address. The list is doubly-linked, allowing efficient insertion and removal.

  mru_init(): Initializes the MRU list pointers (mru_head and mru_tail) and the spinlock. Called at kernel startup.

  mru_touch(int pid, uint64 vpn, char *kva)

- Updates the MRU list when a page is accessed:
  - If the page is already in the list: Moves it to the head, marking it as the most recently used.
  - If the page is new: Allocates a new mru_entry and inserts it at the head.
- Ensures the MRU ordering reflects recent page accesses.

mru_pop_mru()

- Removes and returns the head of the MRU list, i.e., the most recently used page.
- Used during memory allocation failures to select a victim page for eviction.

dump_mru_list()

- Iterates through the MRU list and prints process ID and virtual page number.
- Helps verify MRU ordering and page replacement behavior.

Purpose:

- Keeps track of the most recently used pages globally.
- Provides functions to update page access, evict pages, and debug MRU state.
- Works together with trap.c and vm.c to implement MRU-based demand paging.

2. 3. sysproc.c

```c
#define SBRK_EAGER 1

extern struct proc* get_proc_table(void);
extern struct spinlock* get_ptable_lock(void);
extern struct {
  struct spinlock lock;
  struct proc proc[NPROC];
} ptable;

uint64
sys_getpagestat(void) {
    int pid;
    struct pagestat *uptr;

    argint(0, &pid);
    if(argptr(1, (void*)&uptr, sizeof(*uptr)) < 0) return -1;

    struct proc *p;
    acquire(&ptable.lock);
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->pid == pid){
            struct pagestat copy = p->pst; // <- is pst ever filled?
            release(&ptable.lock);
            if(copyout(myproc()->pagetable, (uint64)uptr, (char*)&copy, sizeof(copy)) < 0)
                return -1;
            return 0;
        }
    }
    release(&ptable.lock);
    return -1;
}

uint64
sys_dumpmru(void)
{
  dump_mru_list();
  return 0;
}
```

- sys_getpagestat():
    - Reads pagestat of a given process and copies to user space.
- sys_dumpmru():
    - Prints MRU list for debugging/monitoring.
- Purpose: Provides system calls for monitoring paging activity.

2. 4. syscall.c / syscall.h

```c
#define SYS_getpagestat  22
#define SYS_dumpmru      23
```

```c
extern uint64 sys_getpagestat(void);
extern uint64 sys_dumpmru(void);
```

```c
[SYS_getpagestat] sys_getpagestat,
[SYS_dumpmru] sys_dumpmru,
```

- Defines syscall numbers: 22 and 23.
- Purpose: Connects kernel syscalls to user programs.

2. 5. usys.pl / user.h

Declares user-level functions:

```
int getpagestat(int pid, struct pagestat *st);
int dumpmru(void);
```

Usys.pl:

```
45 entry("getpagestat");
46 entry("dumpmru");
```

Purpose: Allows user programs to call the new system calls.

2. 6. mrumem.c (User Test Program)

- Allocates more memory than physical RAM using sbrk().
- Randomly accesses pages to trigger page faults.
- Periodically calls getpagestat() to print page faults, swap-ins, swap-outs.
- Calls dumpmru() to verify MRU ordering.
- Purpose: Tests and demonstrates correctness of MRU demand paging

Purpose: Exposes kernel functions to multiple files.

2. 7. proc.c

```
struct {
  struct spinlock lock;
  struct proc proc[NPROC];
} ptable;

struct proc* get_proc_table(void) {
  return ptable.proc;
}

struct spinlock* get_ptable_lock(void) {
  return &ptable.lock;
}
struct proc*
getproc_by_pid(int pid)
{
  struct proc *p;
  acquire(&ptable.lock);
  for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
    if(p->pid == pid){
      release(&ptable.lock);
      return p;
    }
  }
  release(&ptable.lock);
  return 0;
}
```

- Implements helper functions:

o getproc_by_pid(int pid) → find process by pid. o
get_proc_table() → returns process table pointer. o
get_ptable_lock() → returns ptable spinlock.

- Purpose: Supports kernel code in swapping and MRU management.

2.8 Makefile



Compiled user test _mrumem and swap.o.

2.9 OUTPUT:

```
Allocated virtual base at 0x000000000010000
Writing across all pages to force demand faults...
→ Page faults handled successfully.
Stats: PageFaults=64, SwapIns=12, SwapOuts=53

------------------------------------------------
   MRU List  (Most Recently Used → Least)
   [total=12 pages tracked]
------------------------------------------------
  1. pid=6  |  va=0x07000
  2. pid=7  |  va=0x08000
  3. pid=9  |  va=0x10000
  4. pid=9  |  va=0x0f000
  5. pid=8  |  va=0x0e000
  6. pid=8  |  va=0x0c000
  7. pid=7  |  va=0x0b000
  8. pid=6  |  va=0x05000
  9. pid=6  |  va=0x04000
 10. pid=5  |  va=0x03000
 11. pid=5  |  va=0x02000
------------------------------------------------

[TEST 2] Data Consistency Verification
Validating written data across swapped pages...
→ All 64 pages verified. No corruption detected.

Final Summary:
PageFaults=112  |  SwapIns=35  |  SwapOuts=81

------------------------------------------------
   MRU List  (Most Recently Used → Least)
   [total=12 pages tracked]
------------------------------------------------
  1. pid=4  |  va=0x01000
  2. pid=9  |  va=0x10000
  3. pid=6  |  va=0x07000
  4. pid=7  |  va=0x08000
  5. pid=9  |  va=0x0f000
  6. pid=8  |  va=0x0e000
  7. pid=8  |  va=0x0c000
  8. pid=7  |  va=0x0b000
  9. pid=6  |  va=0x05000
 10. pid=6  |  va=0x04000
 11. pid=5  |  va=0x03000
------------------------------------------------

All MRU Tests Completed Successfully ✅
System ready for next command.
```